APUNTES. LAS INTERFACES en C#

Programación orientada a objetos con .NET – Teoría + práctica

1. ¿Qué es una interfaz?

Una interfaz en C# es un "contrato": define qué métodos, propiedades o eventos debe tener un objeto, pero no dice cómo se hacen por dentro. Es como decir: "si quieres ser de este tipo, debes poder hacer esto".

En resumen, una interfaz sirve para:

- Definir un conjunto de capacidades comunes (por ejemplo: "se puede imprimir", "se puede guardar").
- Programar contra abstracciones: tu código depende del contrato, no de una clase concreta.
- Permitir que clases distintas se comporten igual desde el punto de vista del programa.
- Facilitar el mantenimiento, pruebas (testing) y el cambio de implementaciones sin romper el código.

Consejo: Piensa en una interfaz como un enchufe: la forma del enchufe (interfaz) es la misma, pero el aparato (clase) que conectas puede ser distinto.

2. ¿Qué puede contener una interfaz?

En C# moderno, una interfaz puede declarar:

- Métodos (por ejemplo: void Guardar())
- Propiedades (por ejemplo: string Nombre { get; })
- Eventos
- Indexadores
- Y (desde C# 8) incluso implementaciones por defecto, aunque en este módulo nos centraremos en lo básico.

Ojo: Una interfaz no se instancia con new. Primero la implementas en una clase, y luego creas la clase.

3. Cómo se declara e implementa

Se declara con la palabra clave interface, y se implementa en una clase usando dos puntos (:).

```
public interface ISaludable
{
    void Saludar();
}

public class Persona : ISaludable
{
    public void Saludar()
    {
        Console.WriteLine("Hola!");
    }
}
```

Fíjate en dos cosas:

1. 1) La interfaz solo define el método. 2) La clase está obligada a implementarlo.

4. Ejemplo 1: "Imprimible"

Objetivo: crear una interfaz con un método sencillo y una clase que lo implemente.

```
using System;

public interface IImprimible
{
    void Imprimir();
}

public class Ticket: IImprimible
{
    public void Imprimir()
    {
        Console.WriteLine("=== TICKET ===");
        Console.WriteLine("Gracias por su compra.");
    }
}

public class Program
{
    public static void Main()
    {
        Ticket t = new Ticket();
        t.Imprimir();
    }
}
```

Explicación:

• IImprimible obliga a tener el método Imprimir().

• Ticket cumple el contrato y escribe por consola.

• En Main creamos un Ticket y lo imprimimos.

5. Ejemplo 2: polimorfismo con interfaz

Objetivo: tener varias clases distintas que se tratan igual porque comparten la misma interfaz.

```
using System;
using System.Collections.Generic;

public interface INotificable
{
    void Notificar(string mensaje);
}

public class Email: INotificable
{
    public void Notificar(string mensaje)
    {
        Console.WriteLine($"[EMAIL] {mensaje}");
    }
}

public class Sms: INotificable
{
    public void Notificar(string mensaje)
    {
        Console.WriteLine($"[SMS] {mensaje}");
    }
}

public class Program
{
    public static void Main()
    {
        List<INotificable> canales = new List<INotificable>
        {
            new Email(),
```

```
            new Sms()
        };

        foreach (INotificable canal in canales)
        {
            canal.Notificar("Pedido enviado");
        }
    }
}
```

¿Qué estás practicando aquí?

• Una lista puede guardar objetos diferentes (Email y Sms) si ambos cumplen INotificable.

• El foreach llama a Notificar() sin saber qué clase concreta hay dentro: eso es polimorfismo.

6. Ejemplo 3: repositorio de Provincias

Objetivo: usar una interfaz para desacoplar la lógica del programa del acceso a datos. Así puedes cambiar SQL Server por otra fuente (memoria, fichero, API) sin reescribir el menú.

Paso 1: definimos el modelo y la interfaz del repositorio.

```
public class Provincia
{
    public int IdProvincia { get; set; }
    public string Nombre { get; set; } = "";
}

public interface IProvinciaRepository
{
    List<Provincia> GetAll();
    void Add(string nombre);
    bool Update(int id, string nuevoNombre);
    bool Delete(int id);
}
```

Paso 2: una implementación “en memoria” para practicar sin base de datos.

```
using System;
using System.Collections.Generic;
using System.Linq;
```

```
public class ProvinciaRepositoryMemoria : IProvinciaRepository
{
    private readonly List<Provincia> _data = new()
    {
        new Provincia { IdProvincia = 1, Nombre = "Álava" },
        new Provincia { IdProvincia = 2, Nombre = "Sevilla" }
    };

    public List<Provincia> GetAll() => _data.OrderBy(p =>
p.IdProvincia).ToList();

    public void Add(string nombre)
    {
        int nuevoId = _data.Count == 0 ? 1 : _data.Max(p =>
p.IdProvincia) + 1;
        _data.Add(new Provincia { IdProvincia = nuevoId, Nombre = nombre
});
    }

    public bool Update(int id, string nuevoNombre)
    {
        Provincia? p = _data.FirstOrDefault(x => x.IdProvincia == id);
        if (p == null) return false;
        p.Nombre = nuevoNombre;
        return true;
    }

    public bool Delete(int id)
    {
        Provincia? p = _data.FirstOrDefault(x => x.IdProvincia == id);
        if (p == null) return false;
        _data.Remove(p);
        return true;
    }
}
```

Paso 3: el programa usa la interfaz, no la clase concreta.

```
using System;

public class Program
{
    public static void Main()
    {
```

```
        IProvinciaRepository repo = new ProvinciaRepositoryMemoria();

        Console.WriteLine("Provincias iniciales:");
        foreach (var p in repo.GetAll())
            Console.WriteLine($"{p.IdProvincia} - {p.Nombre}");

        repo.Add("Madrid");

        Console.WriteLine("\nTras insertar Madrid:");
        foreach (var p in repo.GetAll())
            Console.WriteLine($"{p.IdProvincia} - {p.Nombre}");
    }
}
```

¿Por qué es más completo? Porque:

- Tu programa depende de IProvinciaRepository (contrato).
- Hoy usas ProvinciaRepositoryMemoria, mañana puedes usar ProvinciaRepositorySql sin cambiar el menú.
- Es más fácil probar (testing): puedes probar la lógica con memoria sin necesitar SQL Server.

7. Errores típicos con interfaces

- Confundir interfaz con clase: una interfaz no lleva estado “normal” (campos) ni se instancia con new.
- Olvidar implementar un método: el compilador te avisa, porque el contrato no se cumple.
- Usar la clase concreta en lugar de la interfaz: pierdes la ventaja de poder cambiar implementaciones.
- No poner el namespace correcto o no añadir using necesarios en el archivo.

8. Resumen

- Interfaz = contrato.
- Sirve para desacoplar y hacer código flexible.
- Permite polimorfismo: tratar objetos distintos de forma uniforme.
- Muy útil para repositorios (BBDD), servicios y pruebas.